

PALANTIR FOUNDRY PROJECT ENABLEMENT PROGRAM

Phase 1 — Day 2

Module 1: OSDK Architecture

2 Hours • Day 2 of 10 • Foundation Accelerator

Presented by Rajesh Pasham

What You Built on Day 1

- Vehicle object type — 8 properties, backed by real data
- Transponder object type — 6 properties, backed by real data
- A one-to-one link: assignedTransponder

Today, we access all of this from code for the first time — via the OSDK

What Is the Ontology SDK (OSDK)?

- A generated, typed client library for your specific Ontology
- Available for TypeScript, Python, and Java, with OpenAPI specs for anything else
- Treats Foundry as your backend — read and write the Ontology with minimal boilerplate
- Regenerated whenever your Ontology changes, so your code always matches your object types

If your Ontology has a Vehicle object type, the OSDK gives you a Vehicle class with methods to query and act on it

Why Not Just Call the REST API Directly?

- Foundry's platform APIs are available as raw REST endpoints — you could call them directly
- The OSDK adds a typed layer on top: property names, types, and action parameters are all checked at compile time
- Editor autocomplete replaces guessing endpoint URLs and payload shapes

Net effect: fewer runtime surprises, faster iteration, safer refactors

Where the OSDK Sits



*OSDKs add ergonomic, typed abstractions on top of the generic platform SDKs — you work with *Vehicle* and *Transponder*, not raw *REST* calls.*

What's Inside a Generated OSDK Package

- objects/

One typed class per object type — Vehicle, Transponder

- actions/

One typed function per action type — markVehicleForMaintenance

- queries/

One typed function per query, once you define any

- client.ts (yours, not generated)

Wires authentication to the generated types

Developer Console

- Where OSDK applications and OAuth clients are created and managed
- A Custom Application bundles:

An OAuth client (public or confidential)

A generated OSDK package, scoped to the Ontology resources you select

Optional web hosting for frontend-only apps

- Generates API documentation (TypeScript, Python, cURL) specific to your objects, actions, and functions

Creating Your First Application

- In Developer Console, select + New application
- Choose Client-facing application for a browser-based app
- On the Permissions page, use the Authorization code grant and set the redirect URL (e.g. <http://localhost:8080/auth/callback>)
- Add Vehicle and Transponder to the application's Ontology resources
- Foundry generates a TypeScript (or Python/Java) SDK package scoped to exactly those resources

Three Ways to Authenticate

- Public OAuth client

Browser apps; user logs in interactively via Foundry's login flow (Authorization Code + PKCE)

- Confidential OAuth client

Server-to-server; a client ID and secret exchange for a token — no user login

- Token-based (user or service token)

Simplest for local scripts and CLIs — a long-lived token stands in for a session

- Today's hands-on: public client for the frontend, and a quick token-based script in Python

Public OAuth Client — TypeScript

```
import { createPublicOAuthClient } from "@osdk/oauth";
import { createClient } from "@osdk/client";

const auth = createPublicOAuthClient(
  "<YOUR_CLIENT_ID>",
  "<YOUR_FOUNDRY_URL>",
  "http://localhost:8080/auth/callback",
);

export const client = createClient(
  "<YOUR_FOUNDRY_URL>",
  "<YOUR_ONTOLOGY_RID>",
  auth,
);
```

Confidential OAuth Client — When to Use It

- No user is present — a backend service or scheduled job needs its own identity
- A client ID and client secret are exchanged directly for an access token
- Well suited to server-to-server integrations, automated jobs, and API integrations with external platforms

Later sprints: this is the pattern used for scheduled polling against UPS and Neology

Confidential OAuth Client — TypeScript

```
import { createConfidentialOAuthClient } from "@osdk/oauth";
import { createClient } from "@osdk/client";

const auth = createConfidentialOAuthClient(
  "<YOUR_CLIENT_ID>",
  process.env.OSDK_CLIENT_SECRET!,
  "<YOUR_FOUNDRY_URL>",
);

export const client = createClient(
  "<YOUR_FOUNDRY_URL>",
  "<YOUR_ONTOLOGY_RID>",
  auth,
);

// Never commit OSDK_CLIENT_SECRET — load it from
// an environment variable or secrets manager.
```

Token-Based Auth — When to Use It

- The fastest path for local scripts, notebooks, and CLIs
- A long-lived user or service token stands in for a full session — no OAuth redirect flow at all
- Used by today's Python quick-start (UserTokenAuth)

Trade-off: tokens are powerful and long-lived, so they need the same care as a password

Reading Objects With the OSDK

- `client(Vehicle).fetchPage()` — read a page of objects
- `.where({ status: { $eq: "Active" } })` — filter objects
- `.fetchPage({ $orderBy: { year: "desc" } })` — sort results
- `.iterate()` — stream through every matching object

Every method is fully typed against your Vehicle object type — your editor autocompletes property names

Fetching Vehicles — TypeScript

```
import { Vehicle } from "@your-app/sdk";
import { client } from "../auth";

async function listActiveVehicles() {
  const page = await client(Vehicle)
    .where({ status: { $eq: "Active" } })
    .fetchPage({ $orderBy: { year: "desc" } });

  for (const vehicle of page.data) {
    console.log(vehicle.vehicleId, vehicle.make, vehicle.model);
  }
}
```

Pagination Deep Dive

- `fetchPage()` returns one page plus a `nextPageToken` when more results exist
- Pass { `$nextPageToken` } back into `fetchPage()` to retrieve the following page
- `.iterate()` wraps this loop for you — use it when you want every matching object

Use `fetchPage()` when you control page size for a UI list; use `iterate()` for background jobs

Error Handling

```
try {
  const page = await client(Vehicle).fetchPage();
  // use page.data
} catch (error) {
  if (isOsdkError(error)) {
    console.error(error.errorName, error.parameters);
  } else {
    throw error;
  }
}

// isOsdkError is exported from @osdk/client
```

Writing Data — Action Types

- Every write to the Ontology goes through an Action Type — never a direct object edit
- Actions are generated into the OSDK just like objects — typed parameters, typed return values
- Queries are read-only, reusable server-side logic — useful for aggregations or complex lookups

Today: we define a simple `markVehicleForMaintenance` action and call it from code

Calling an Action — TypeScript

```
import { markVehicleForMaintenance } from "@your-app/sdk";
import { client } from "../auth";

async function sendToMaintenance(vehicleId: string) {
  await client(markVehicleForMaintenance).applyAction({
    vehicle: vehicleId,
    status: "Maintenance",
  });
}

// Exact parameter names come from your generated SDK's
// API Documentation tab in Developer Console.
```

Queries vs. Actions

	Query	Action
Purpose	Read-only computation	Write to the Ontology
Changes data?	Never	Always, via a Function
Typical use	Aggregations, complex lookups	Create / update / delete an object
Called from OSDK as	<code>client(myQuery).executeFunction()</code>	<code>client(myAction).applyAction()</code>

Anatomy of an OSDK Object Instance

- Every fetched object exposes its properties directly — `vehicle.make`, `vehicle.status`
- `$primaryKey` and `$title` give you the object's key and display title without guessing which property holds them
- `$apiName` identifies which object type an instance came from — useful in generic UI code

These \$-prefixed fields are metadata added by the OSDK, not properties you defined

Quick-Start With Python

```
from foundry_sdk import FoundryClient
from foundry_sdk.auth import UserTokenAuth

auth = UserTokenAuth(hostname="<YOUR_ENROLLMENT>.palantirfoundry.com")
client = FoundryClient(auth=auth, hostname="<YOUR_ENROLLMENT>.palantirfoundry.com")

for vehicle in client.ontology.objects.Vehicle.where(
    client.ontology.objects.Vehicle.status == "Active"
).iterate():
    print(vehicle.vehicleId, vehicle.make, vehicle.model)
```

Keeping Credentials Safe

- Never commit a client secret or a personal token to source control
- Load secrets from environment variables or a secrets manager, not hard-coded strings
- Public OAuth clients don't have a secret to leak — that's exactly why browser apps use them

Add .env to .gitignore before your first commit today

OSDK Glossary Cheat-Sheet

- OSDK

Generated, typed client library for your Ontology

- Developer Console

Where OSDK applications and OAuth clients are created

- Client ID / Redirect URL / Ontology RID

The three values every OSDK client needs

- applyAction / fetchPage / iterate

The core OSDK methods for writing and reading

Day 2 Troubleshooting

- “The browser login loop never completes.”

Double-check your redirect URL matches exactly what's configured in Developer Console.

- “my action call is rejected.”

Re-check exact parameter names in your generated API Documentation — they're case-sensitive.

- “TypeScript can't find my generated package.”

Re-run the install command shown on your application's SDK tab — it changes per application.

Today's Definition of Done

- A Developer Console application exists, scoped to Vehicle, Transponder, and markVehicleForMaintenance
- Your local TypeScript project authenticates via the public OAuth client
- Your script prints all Active vehicles, sorted by year
- Calling sendToMaintenance updates a real Vehicle's status, visible in Ontology Manager

(Stretch) The Python script reproduces the read using token-based authentication

What You'll Build in the Next 45 Minutes

- Create a Developer Console application and generate your first OSDK package
- Set up a local TypeScript project and authenticate with a public OAuth client
- Fetch and filter your Vehicle objects from Day 1
- Define and call a simple markVehicleForMaintenance action

Stretch goal: repeat the read using the Python quick-start

Full step-by-step instructions are in today's Hands-On Exercise document

Questions?

Open floor — OSDK setup, authentication, or anything from today's walkthrough.

Next Session — Day 3

Module 2: React Application Development

- React architecture, components and hooks
- Routing and responsive layout
- State management and Material UI
- *Hands-on lab: develop a mobile-responsive application*